The Google File System –

Sanjay Ghemawat, Howard Gobioff and Shun-Tak Leung

The authors of this paper have designed and implemented a scalable distributed file system for large distributed data-intensive applications called the Google File System (GFS). The GFS provides fault tolerance while running on inexpensive commodity hardware and delivers high aggregate performance to a large number of clients.

The design of the GFS have been driven by key observations of Google applications workloads and technological environment, such as:

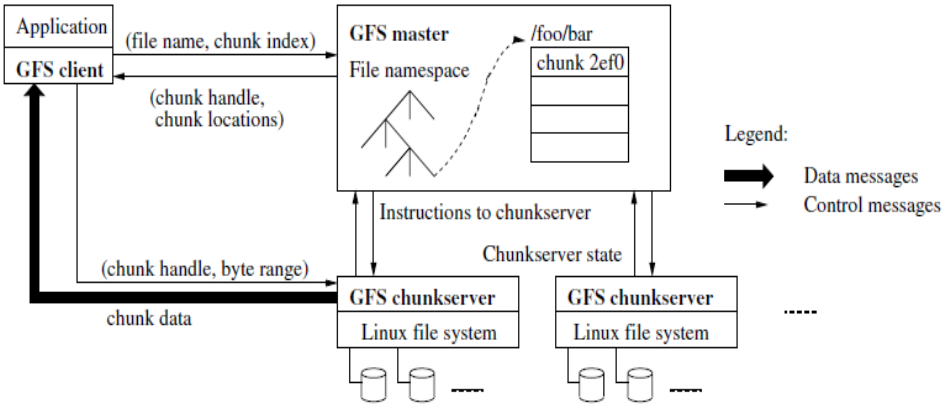
- Constant monitoring, error detection, fault tolerance and automatic recovery must be integral to the system.
- I/O operation and block sizes have to be revisited.
- Appending becomes the focus of performance optimization and atomicity guarantees.
- Co-designing the applications and the file system API benefits the overall system by increasing our flexibility.

Assumptions

- The system is built from many inexpensive commodity components that often fail.
- The system stores a modest number of large files.
- The workloads primarily consist of two kinds of reads: large streaming reads and small random reads.
- The workloads also have many large, sequential writes that append data to files.

- The system must efficiently implement well-defined semantics for multiple clients that concurrently append to the same file.
- High sustained bandwidth is more important than low latency.

Architecture



As shown in the picture, a GFS cluster consists of a single master and multiple chunk servers and is accessed by multiple clients.

Chunk servers store chunks (fixed-sized files) on local disks as Linux files and read or write chunk data specified by a chunk handle and byte range. For reliability, each chunk is replicated on multiple chunk servers.

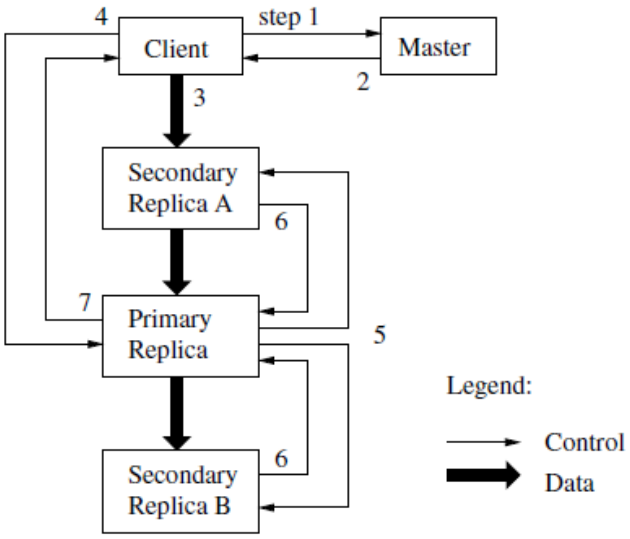
Some of the tasks done by the GFS master are:

- Maintains all file system metadata.
- Make replication decisions using global knowledge.
- Chunk lease management.
- Garbage collection of orphaned chunks.
- Chunk migration between chunk servers.
- Communicate with each chunk server through ‘heartbeat’ messages to give it instructions and collect its state.

The size of each chunk is 64MB. This is larger than the typical file systems block sizes. This offers several advantages, such as:

- Reduces clients need to interact with the master because reads and writes on the same chunk require only one initial request to the master for chunk location information
- Client is more likely to perform many operations on a given chunk reducing network overhead
- Reduces the size of the metadata stored on the master

Write Control and Data Flow



1. The client asks the master which chunk server hold the current lease for the chunk and the locations of the other replicas.
2. The master replies with the identity of the primary and the locations of the other replicas.
3. The client pushes the data to all the replicas.
4. Once all the replicas have acknowledged receiving the data, the client send a write request to the primary.
5. The primary forwards the write request to all secondary replicas.
6. The primary replies to the client.

Conclusions

- GFS demonstrates the qualities essential for supporting large-scale data processing workloads on commodity hardware.
- Observations have led to radically different points in the design space.
- Authors treat component failures as the norm rather than the exception.
- GFS system provides fault tolerance by constant monitoring, replicating crucial data and fast and automatic recovery.
- Use check summing to detect data corruption at the disk or IDE subsystem level.
- Design delivers high aggregate throughput to many concurrent readers and writers performing a variety of tasks.
- GFS has successfully met the storage needs in Google and is widely used within the company as the storage platform for research and development as well as production data processing.

Questions

The following questions should be answer after analyzing any scientific paper.

1. What is the problem that arises in the paper?

   The authors work at one of the biggest technology companies in the world and they have seen there is a rapidly growing demand of Google's data processing need. This inspires the authors to take a look into the problem and analyze the existing distributed file systems.

2. Why is this an interesting or important problem?

   Due to the exponential increase of data there is a rapidly growing demand of data processing need. This is a worldwide problem and not only a problem at Google. Therefore, any solution to this problem is going to be helpful to many companies. This is why the solution proposed is interesting and relevant.

3. What are other solutions that have been proposed to solve this problem?

Solutions proposed to solve this problem include previous distributed file systems such as AFS, xFS, Frangipani and Intermezzo.

4. What is the solution the authors propose?

The authors designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients

5. How successful is this solution?

The fact that this implementation is used in one of the world's biggest technology companies means that the solution proposed by the authors in this paper is very successful and it can be implemented by other companies around the world. Sometimes we read many papers proposing solutions to different kind of problems. This papers shows us experiments and results on how the solutions being proposed can solve the problem in discussion but not all the papers shows us a real-world example of the solution being implemented. In this case, the file system proposed by the authors is widely use in real-world scenarios which makes the solution and the paper itself more interesting and relevant.